Thứ ngày tháng năm

Bài 1:

Lý Tú Anh
13 DHCNTT5
18DDS0005

```
interface Mail {
    void sendMail ();
    void log ();
}

class MailImpl implements Mail {
    @Override
    public void sendMail () {
        System.out.println ("Đang gửi mail");
    }

    @Override
    public void log () {
        System.out.println ("Đang nhập : ");
    }
}

class Main {
    public static void main (String [] args) {
        MailImpl mail = new MailImpl ();
        mail.sendMail ();
        mail.log ();
    }
}
```

Thứ ngày tháng năm

Bài 2

```
interface Shape {
    void draw ();
    ~~Sh~~ String color = "Red";
} ~~class~~
public class Rectangle ~~extends~~ implements Shape {
    ~~class~~ @ Override
    public void draw () {
        System.out.println (" Hình màu "+color);
    }
}

public class Circle implements Shape {
    @ Override
    public void draw () {
        System.out.println (" Hình tròn màu "+color);
    }
}

class Main {
    public static void main (String [] args) {
        Shape s1 = new Rectangle ();
        Shape s2 = new Circle ();
        ~~draw~~ s1.draw ();
        ~~draw~~ s2.draw ();
    }
}
```

```
class Main {
    public static void main(String[] {
        Person p = new Person(String);
        MyBoy m = new MyBoy("Jake");
        MyGirl g = new MyGirl("Kate");
        System.out.println("Name: " + p.getName());
        System.out.println("Name: " + m.getName());
        System.out.println("Name: " + g.getName());
    }
}

abstract class L {
    void noise(String);
    String getFriend();
    double getLength();
}

@Override
public void noise() {
    System.out.println("Ha" + noise + ");
}

@Override
public void toy() {
    System.out.println("Ha");
}

@Override
public void toy() {
    System.out.println("Ha");
}

@Override
public void toy() {
    System.out.println("Ha");
}

@Override
public double getLength() {
    return 10;
}

class MyBoy implements MyGirl {
    private String friend;
    private int length;
    public MyBoy(String name) {
        super(name);
        this.friend = friend;
        this.length = length;
    }
    @Override
    public void toy() {
        System.out.println("MyBoy toy");
    }
    @Override
    public void toy() {
        System.out.println("MyGirl toy");
    }
    @Override
    public void toy() {
        System.out.println("MyGirl toy");
    }
    @Override
    public double getLength() {
        return 5.0;
    }
}

class Person implements MyGirl {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    @Override
    public void toy() {
        System.out.println("Person toy");
    }
    @Override
    public void toy() {
        System.out.println("Person toy");
    }
    @Override
    public void toy() {
        System.out.println("Person toy");
    }
    @Override
    public double getLength() {
        return 9.0;
    }
}

interface Shape {
    void draw();
    void move(int, int);
    void toy();
}

abstract class MyShape implements Shape {
    protected int x, y;
    protected int color;
    protected boolean visible;
    protected MyShape(int x, int y, int color) {
        this.x = x;
        this.y = y;
        this.color = color;
        this.visible = true;
    }
    @Override
    public void move(int dx, int dy) {
        x += dx;
        y += dy;
        draw();
    }
    @Override
    public void toy() {
        System.out.println("Toy");
    }
    @Override
    public void draw() {
        System.out.println("Drawing " + this.toString());
    }
    @Override
    public void toy() {
        System.out.println("Toy");
    }
    @Override
    public void toy() {
        System.out.println("Toy");
    }
    @Override
    public double getLength() {
        return 10.0;
    }
}

class Circle implements Shape {
    private int radius;
    public Circle(int x, int y, int radius, int color) {
        super(x, y, color);
        this.radius = radius;
    }
    @Override
    public void draw() {
        System.out.println("Circle at (" + x + "," + y + ") radius=" + radius);
    }
    @Override
    public void toy() {
        System.out.println("Circle toy");
    }
    @Override
    public double getLength() {
        return 2 * Math.PI * radius;
    }
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o instanceof Circle c) {
            return radius == c.radius && x == c.x && y == c.y;
        }
        return false;
    }
    @Override
    public int hashCode() {
        return Objects.hashCode(x, y, radius);
    }
    @Override
    public void toy() {
        System.out.println("Circle toy");
    }
    @Override
    public void toy() {
        System.out.println("Circle toy");
```

}

bài tập.

```
interface Car {
    void accelarate ();
    string getColor ();
    double getSpeed ();
}

public class Bmw implements Car {
    private string color;
    private double speed;
```

```
~~public Bmw implements Car {~~
    ~~private String color;~~
    public Bmw ( String color , double speed ) {
        this color = color ;
        this . speed = speed ;
    }
    @ Override
    public void accelarate () {
        speed += 20;
        System. out. println ("BMW accelerate...");
    }
    @ Override
    public ~~void~~ String getColor () {
        return color ;
    }
    @ Override
    public double getSpeed () {
        return speed;
    }
    @ Override
    public String to String () {
        return "BMW color=" + color + "
Speed = " + speed + "
    }
```

```
public class Civic implements Car {
    private String color ;
    private double speed ;
    public civic ( String color, double speed ) {
        this color = color;
        this speed = speed;
    }
    @ Override
    public void accelerate () {
        speed += 15 ;
        System. out. println ("Civic accelerate ");
    }
    @ Override
    public String getColor ();
        return color ;
    }
    @ Override
    public double getSpeed () {
        return speed ;
    }
    @ Override
    public String to String () {
        return "Civic Color= " + color + " Speed = " + speed + "
    }
```

```
class Main {
    public static void main ( String [] args ) {
        Car c1 = new Bmw ("Red", 0);
        Car c2 = new Civic ("Vang", 0);
        c1. accelerate ();
        c2. accelerate ();
    }
}
```

}

Bank

```
interface Moveable {
    void moveUp ();
    void moveDown ();
    void moveLeft ();
    void moveRight ();
}

class MoveablePoint implements Moveable {
    private int x;
    private int y;
    private int xspeed;
    private int yspeed;
    public MoveablePoint (int x, int y, int xspeed, int yspeed)
        this.x = x;
        this.y = y;
        this.xspeed = xspeed;
        this.yspeed = yspeed;
    }
    @Override
    public void moveUp () {
        y += yspeed;
    }
    @Override
    public void moveDown () {
        y -= yspeed;
    }
    @Override
    public void moveLeft () {
        x -= xspeed;
    }
    @Override
    public void moveRight () {
        x += xspeed;
    }
    @Override
    public String toString () {
        return "point = x = , y = " + y + ",
    xspeed = " + xspeed + ", yspeed = yspeed + ")";
    }

    public class MoveableCircle implements Moveable {
    private int radius;
    private Moveable center;
    public MoveableCircle ( Moveable Point center,
    int radius) {
        this.center = center;
        this.radius = radius;
    }
    @Override
    public void moveUp () {
        center.moveUp ();
    }
    @Override
    public void moveDown () {
        center.moveDown ();
    }
    @Override
    public void moveLeft () {
        center.moveLeft ();
    }
    @Override
    public void moveRight () {
        center.moveRight ();
    }
    @Override
    public String toString () {
        return "Circle { center = " + center + ",
    radius = " + radius + ")";
    }
}

class Main {
    public static void main ( String [] args ) {
    MoveablePoint p = new MoveablePoint (1,1,2,3);
    MoveableCircle c = new MoveableCircle (p,5);
    p.moveRight ();
    p.moveUp ();
    c.moveRight ()
    c.moveUp ();
    }
}
```